MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Student:
Tacu Loredana
FAF-242

REPORT

Laboratory Works No.5 and No.6

The File System with Your Own Hands

Inside the xv6 File System

Operating Systems

Checked by:
Reițman Patricia

UTM
Chișinău – 2026

Purpose of the Laboratory Works

The purpose of Laboratory Work No.5 is to investigate the Linux file-system interface through direct experiments with paths, file types, inodes, links, permissions, storage allocation, mounted devices, and the /proc pseudo-file system. Laboratory Work No.6 continues the same study inside xv6, relating user-visible operations to on-disk structures and kernel code, and implementing two small utilities: mystat and find.

Objectives

1. Distinguish regular files, directories, symbolic links, character devices, and block devices.
2. Explain inode identity, link counts, symbolic-link behavior, directory entries, and sparse-file allocation.
3. Interpret Linux mode bits, umask, directory permissions, set-user-ID, and the sticky bit.
4. Inspect disks, mounts, CPU, memory, process metadata, and file descriptors.
5. Examine xv6 stat, dinode, directory-entry, unlink, and block-reclamation logic.
6. Implement and validate xv6 user programs that display metadata and recursively search directories.

Experimental Environment

The Linux experiments were performed as the unprivileged user loredana on Ubuntu. The xv6-riscv source tree was compiled with riscv64-linux-gnu-gcc and executed with qemu-system-riscv64. The repository was a private working copy of the MIT xv6-riscv tree. All commands and outputs shown in the figures are the results of the completed experiments.

Report Structure

Part I presents the Linux experiments required by Laboratory Work No.5. Part II moves inside xv6 for Laboratory Work No.6. The appendices contain the complete source code of the two implemented utilities, and the report ends with a joint critical analysis and conclusion.

1. Laboratory Work No.5 — The File System with Your Own Hands

1.1 Paths and File Types

Linux exposes several object types through one pathname interface. The first character printed by ls -l identifies the type: d for directory, - for regular file, l for symbolic link, c for character device, and b for block device. The /bin pathname is a symbolic link to /usr/bin on this Ubuntu system; /dev/null is a character device; and /tmp is a directory with the sticky bit set.

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
$ readlink -f /bin
$ find /dev -maxdepth 1 -type c | head
$ find /dev -maxdepth 1 -type b | head
```



Figure 1: Linux path types and examples of character and block devices

Observation

A character device transfers a stream of bytes, while a block device is addressed in blocks and normally backs a file system. Symbolic links are distinct objects whose contents are pathnames; therefore /bin and /usr/bin refer to the same directory after pathname resolution.

1.2 Directory Size and Disk Usage

The size shown for a directory entry is metadata storage, not the recursive size of its contents. The du command traverses the hierarchy and totals allocated blocks. In this environment, /var/log used approximately 1.2 GiB and /usr used approximately 3.5 GiB, making /usr about 2.3 GiB larger and almost three times the size of /var/log.

```
$ ls -ld /var/log /usr
$ du -sh /var/log /usr
```

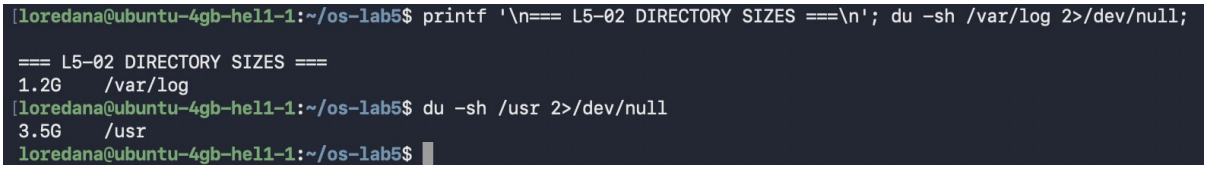


Figure 2: Directory metadata size compared with recursive disk usage

1.3 Inodes and Hard Links

An inode stores a file's metadata and block addresses. A directory maps a name to an inode number. Creating a hard link adds another directory entry for the same inode, so both names initially showed inode 255798 and the link count changed from one to two. Removing one name reduced the count but did not remove the data because the second name still referenced the inode.

```
$ echo hello > hard.txt
$ ls -li hard.txt
$ ln hard.txt hard2.txt
$ ls -li hard.txt hard2.txt
$ rm hard.txt
$ cat hard2.txt
```

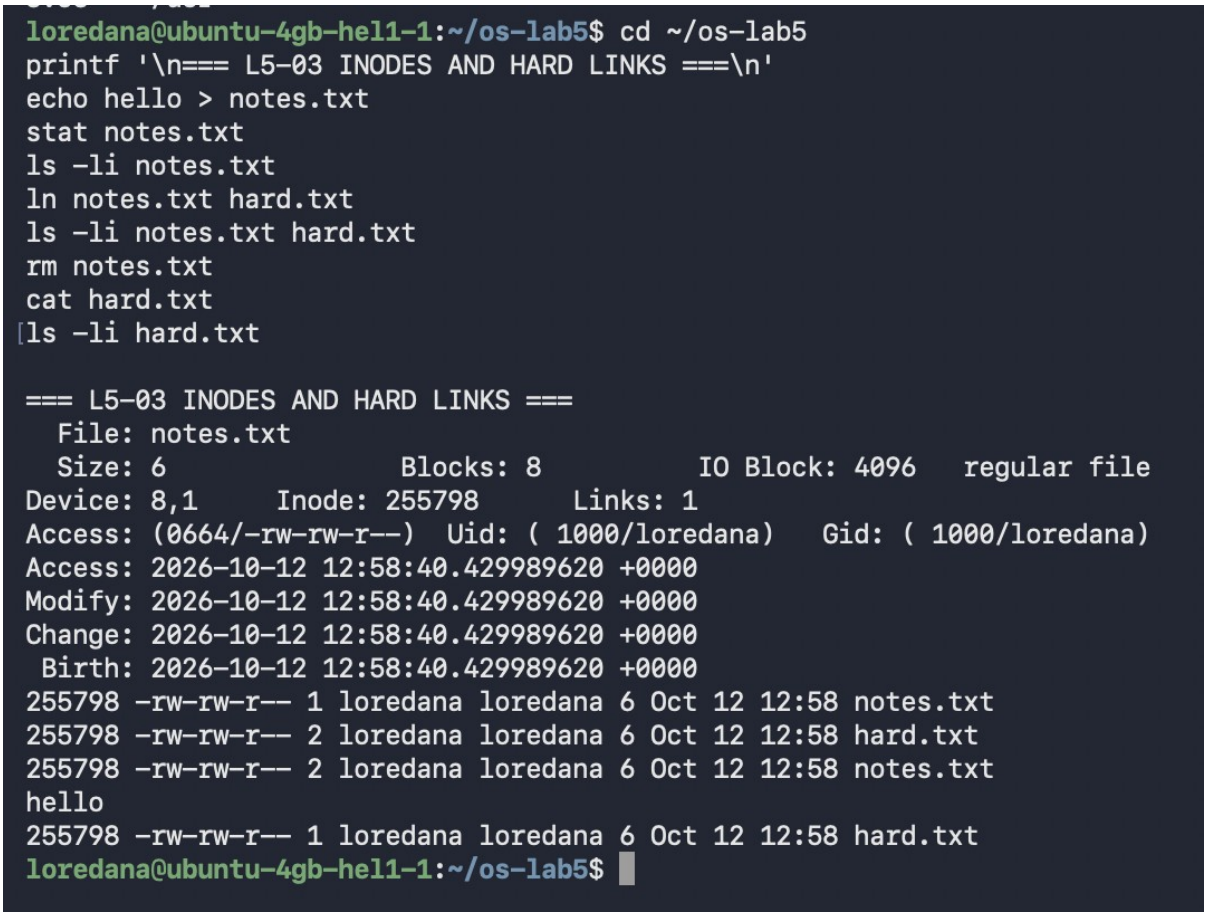


Figure 3: One inode referenced by two hard links

1.4 Symbolic Links and Directory Link Counts

The symbolic link soft.txt had its own inode and stored the eight-character target path hard.txt. Once hard.txt was removed, dereferencing soft.txt failed even though the symbolic-link object remained. A newly created directory had link count two for its parent entry and its own dot entry. Adding subdirectories x and y increased the parent directory's count from two to four because each child contains a .. entry referring to the parent.

```
loredana@ubuntu-4gb-hel1-1:~/os-lab5$ printf '\n=== L5-04 SYMBOLIC AND DIRECTORY LINKS ===\n'
ln -s hard.txt soft.txt
ls -li hard.txt soft.txt
cat soft.txt
rm hard.txt
cat soft.txt
ls -l soft.txt
mkdir d
ls -ld d
mkdir d/x d/y
ls -ld d

=== L5-04 SYMBOLIC AND DIRECTORY LINKS ===
255798 -rw-rw-r-- 1 loredana loredana 6 Oct 12 12:58 hard.txt
255804 lrwxrwxrwx 1 loredana loredana 8 Oct 12 12:59 soft.txt -> hard.txt
hello
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 loredana loredana 8 Oct 12 12:59 soft.txt -> hard.txt
drwxrwxr-x 2 loredana loredana 4096 Oct 12 12:59 d
drwxrwxr-x 4 loredana loredana 4096 Oct 12 12:59 d
loredana@ubuntu-4gb-hel1-1:~/os-lab5$
```

Figure 4: Broken symbolic link and directory link-count growth

1.5 Logical Size versus Allocated Space

File length and allocated space are separate concepts. The two-byte file one.txt occupied one 4 KiB allocation unit. truncate created big.img with a logical length of 1 GiB but no data blocks, so du reported zero allocated space. This sparse representation is useful for disk images and large files containing extensive zero-filled regions.

```
$ echo x > one.txt
$ ls -l one.txt
$ du -h one.txt
$ truncate -s 1G big.img
$ ls -lh big.img
$ du -h big.img
```

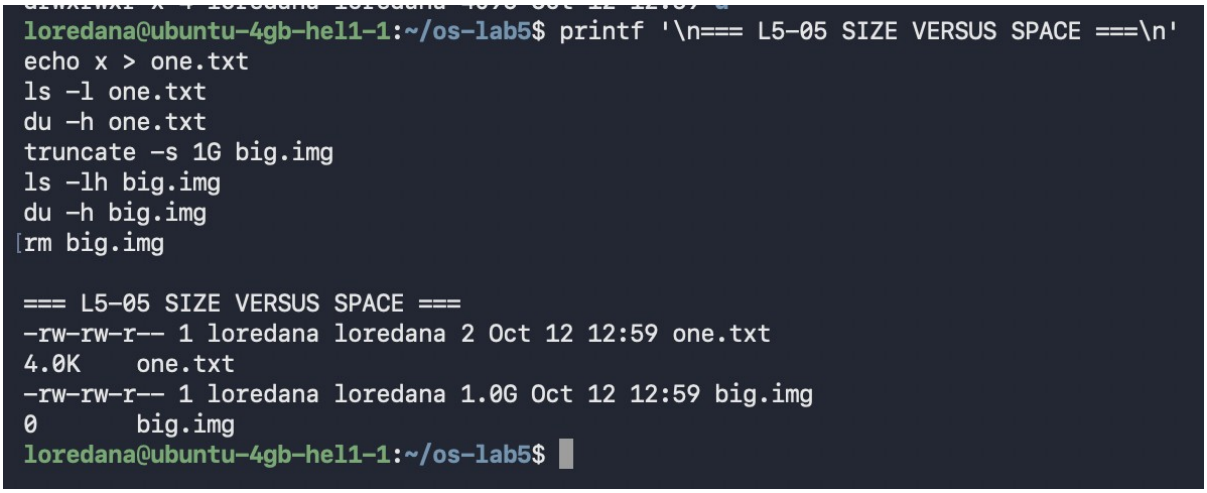


Figure 5: Logical length and allocated space of a sparse file

1.6 File Permissions and umask

Mode 600 grants read and write permission only to the owner. Mode 000 denied reading even to the owner because ownership selects the owner permission triplet but does not bypass it. chmod u+r restored owner read access. With umask 0002, the base modes 666 for files and 777 for directories became 664 and 775 respectively.

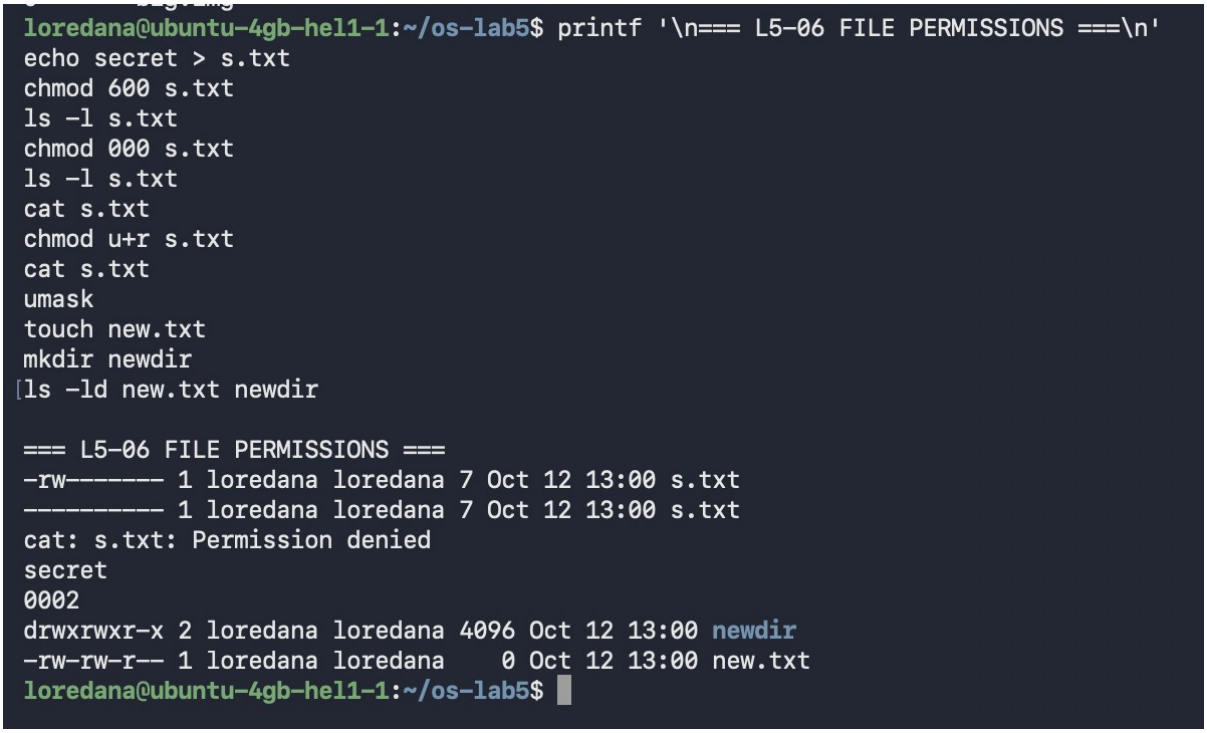


Figure 6: Permission changes, access denial, and umask-derived modes

Recommended Modes

600 is appropriate for private keys and user-only secrets; 644 for ordinary documents readable by others; 700 for private directories or scripts; and 755 for programs and public directories that others must traverse. The least permissive mode that still supports the intended use should be selected.

1.7 Directory Permissions and Special Bits

Read permission on a directory allows listing entry names, whereas execute permission allows pathname traversal and access to known entries. A read-only directory revealed the name f but could not provide its metadata or contents. An execute-only directory could not be listed, yet box/f remained readable because its name was already known.

The set-user-ID bit on /usr/bin/passwd lets it operate with the file owner's effective identity so it can update protected authentication data. The sticky bit on /tmp prevents users from deleting one another's files. Direct reading of /etc/shadow was correctly denied to the ordinary user.

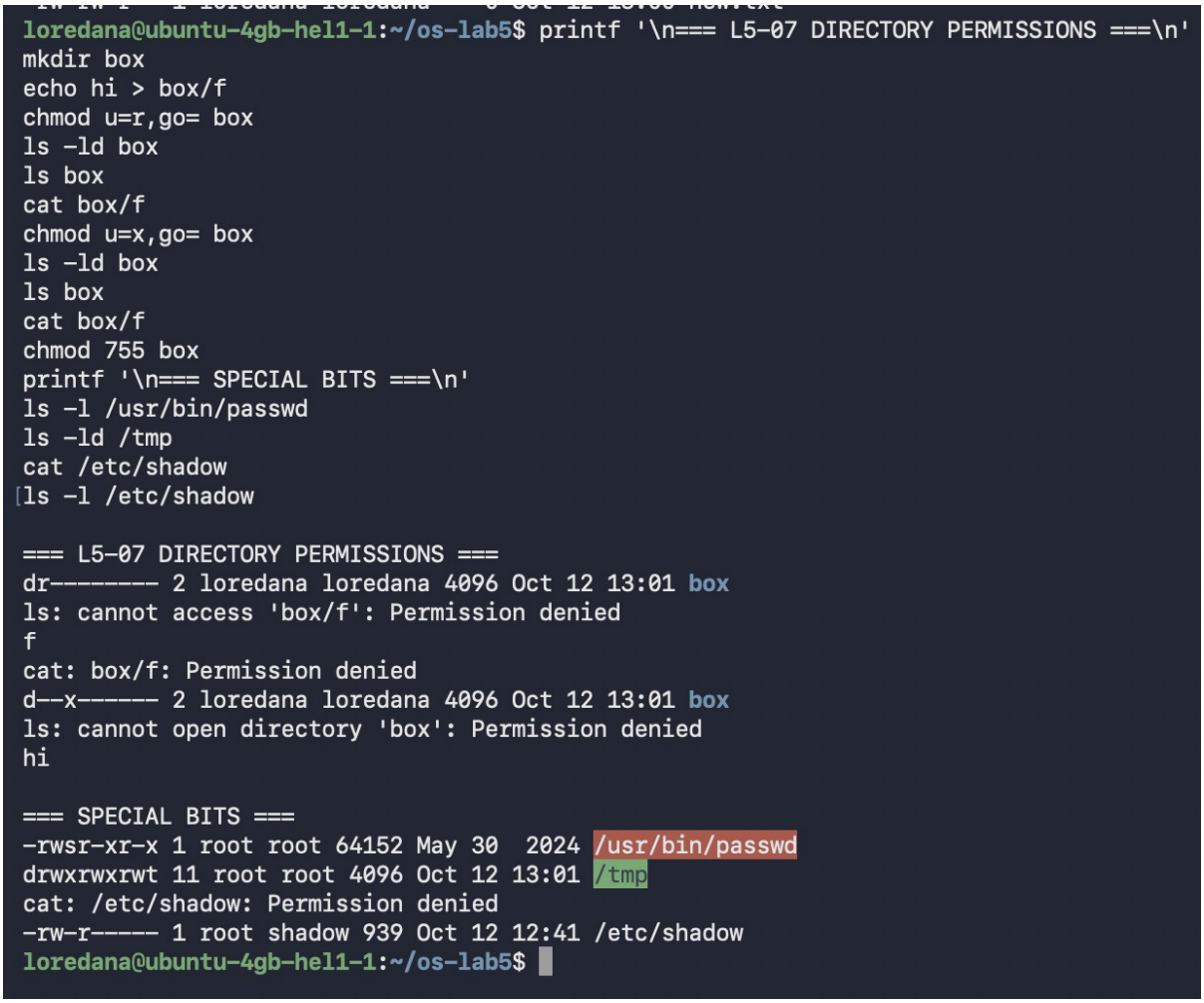


Figure 7: Directory read/execute semantics, set-user-ID, and sticky bit

1.8 Disks and Mounts

The root file system was ext4 on /dev/sda1. At the time of measurement it had a nominal size of 38 GiB, 34 GiB used, and about 2.5 GiB available. df rounded this to 94% use, while lsblk reported 89%; such differences arise from accounting details and reserved space. The EFI system partition was a FAT32 volume on /dev/sda15 mounted at /boot/efi.

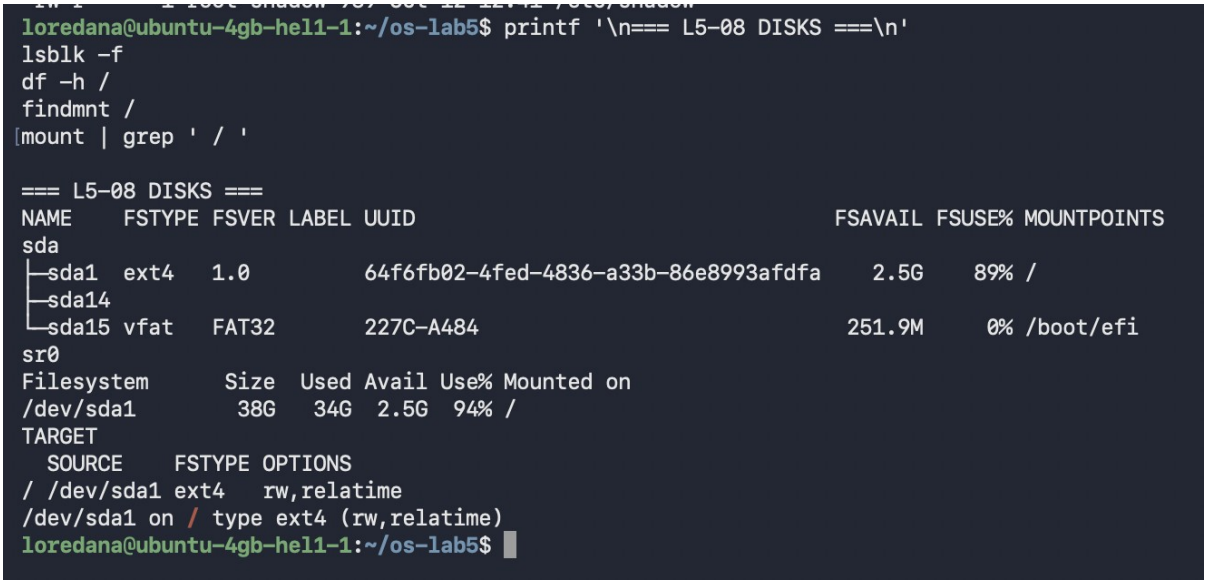


Figure 8: Block devices, root-file-system usage, and mount information

1.9 The /proc Pseudo-File System

The system exposed two processors and 3,911,572 kB of physical memory, with approximately 2.0 GiB available and no swap configured. /proc/self refers dynamically to the process reading it; consequently head observed its own status and its own file-descriptor table. Descriptors 0, 1, and 2 pointed to the terminal, while descriptor 3 referred to the directory being listed.

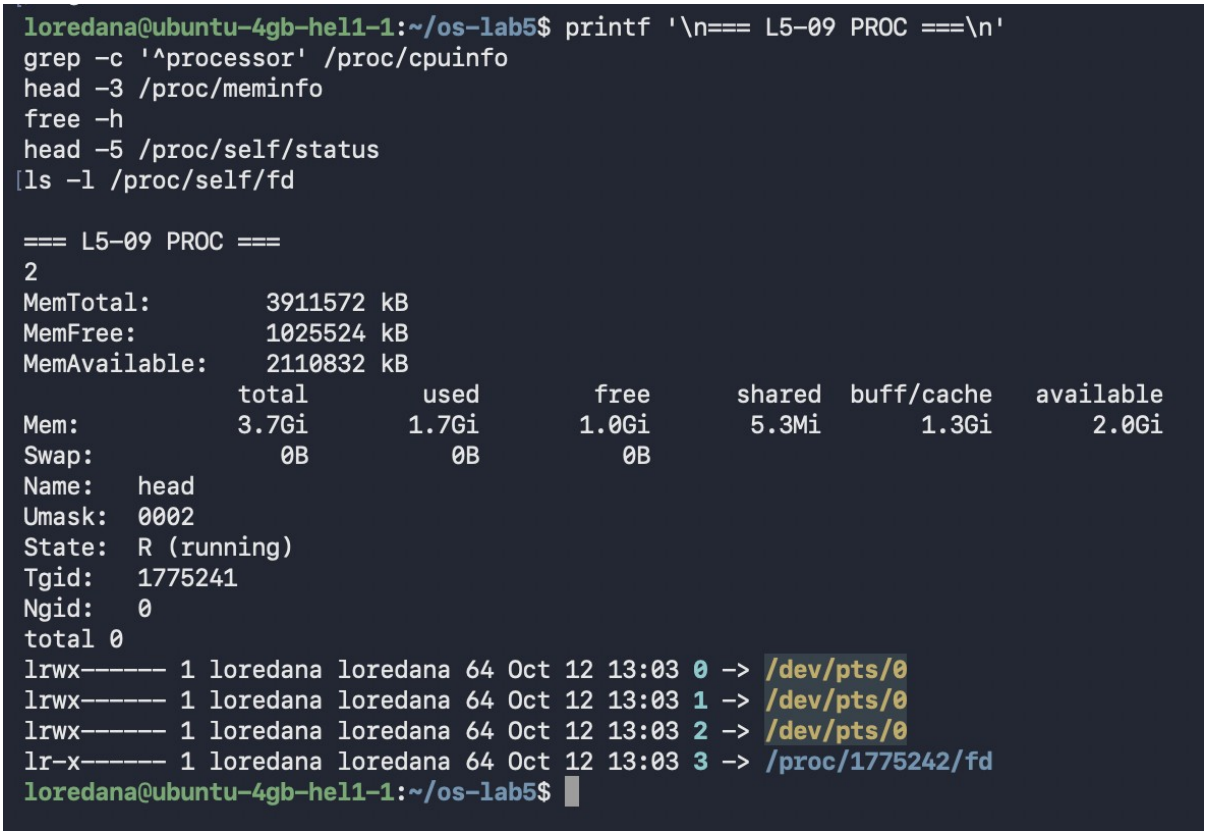


Figure 9: CPU, memory, process status, and open file descriptors in /proc

2. Laboratory Work No.6 — Inside the xv6 File System

2.1 Metadata Structures

xv6 defines three public stat types: T_DIR = 1, T_FILE = 2, and T_DEVICE = 3. struct stat contains the device identifier, inode number, type, link count, and byte size. Its on-disk counterpart, struct dinode, additionally stores major and minor device numbers and an array of direct and indirect block addresses.

```
[loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ printf '\n=== L6-01 STAT TYPES ===\n'; nl -ba kernel/stat.h

=== L6-01 STAT TYPES ===
     1  #define T_DIR    1 // Directory
     2  #define T_FILE   2 // File
     3  #define T_DEVICE 3 // Device
     4
     5  struct stat {
     6    int dev;     // File system's disk device
     7    uint ino;    // Inode number
     8    short type;  // Type of file
     9    short nlink; // Number of links to file
    10    uint64 size; // Size of file in bytes
    11  };
[loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ printf '\n=== L6-01 DINODE FIELDS ===\n'; grep -n -A18 'struct dinode' kernel/fs.h

=== L6-01 DINODE FIELDS ===
32:struct dinode {
33-  short type;              // File type
34-  short major;             // Major device number (T_DEVICE only)
35-  short minor;             // Minor device number (T_DEVICE only)
36-  short nlink;             // Number of links to inode in file system
37-  uint size;               // Size of file (bytes)
38-  uint addrs[NDIRECT + 1]; // Data block addresses
39-};
40-
41-// Inodes per block.
42:#define IPB (BSIZE / sizeof(struct dinode))
43-
44-// Block containing inode i
45-#define IBLOCK(i, sb) ((i) / IPB + sb.inodestart)
46-
47-// Bitmap bits per block
48-#define BPB (BSIZE * 8)
49-
50-// Block of free map containing bit for block b
51-#define BBLOCK(b, sb) ((b) / BPB + sb.bmapstart)
52-
53-// Directory is a file containing a sequence of dirent structures.
54-#define DIRSIZ 14
55-
56-// The name field may have DIRSIZ characters and not end in a NUL
57-// character.
58-struct dirent {
59-  ushort inum;
60-  char name[DIRSIZ] __attribute__((nonstring));
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$
```

Figure 10: xv6 stat types, struct stat, and struct dinode fields

2.2 Root Directory and Directory Entries

The kernel booted successfully with two harts. xv6 assigned inode 1 to the root directory. Both . and .. in the root refer to inode 1 because the root is its own parent. A new empty directory occupied 32 bytes and contained two 16-byte entries: a two-byte inode number and a fourteen-byte name field.

```
$ ls
$ mkdir emptydir
$ ls emptydir
```

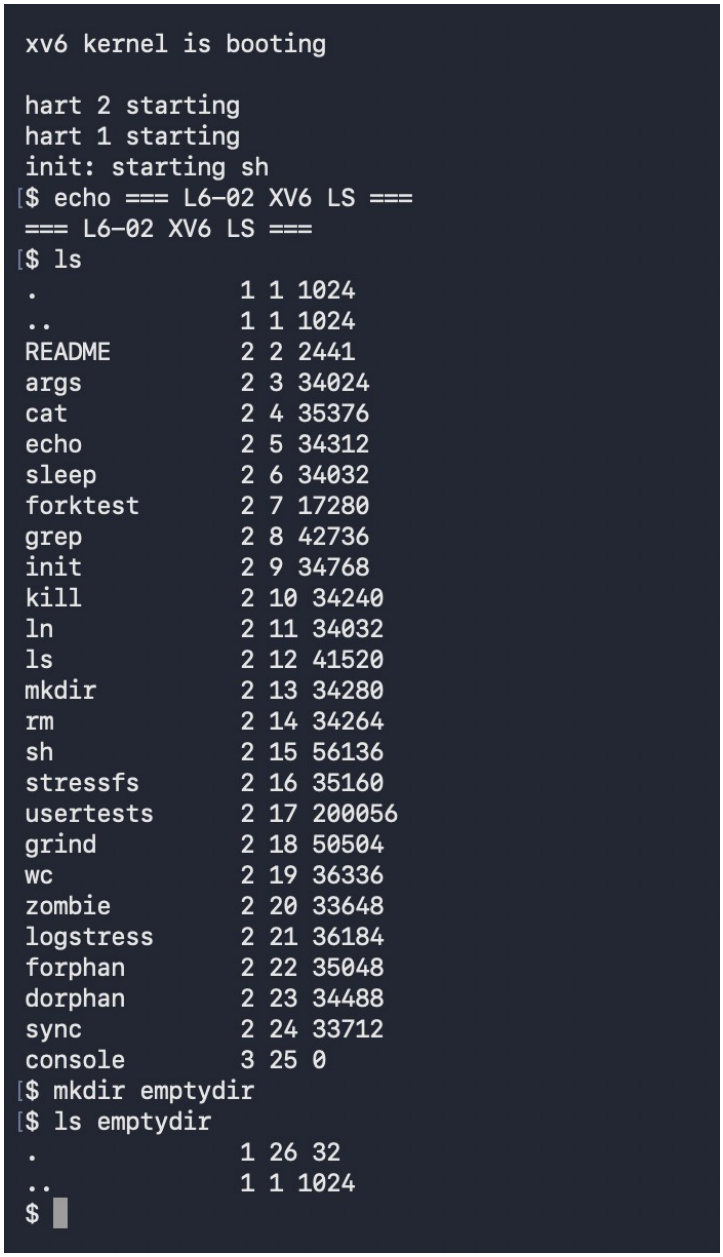


Figure 11: xv6 boot, root listing, and the entries of an empty directory

2.3 Registering User Programs

The programs were added to the UPROGS list as $U/_mystat and $U/_find. This causes their user-space binaries to be linked and copied into fs.img by mkfs, making them available in the xv6 shell after the next build.

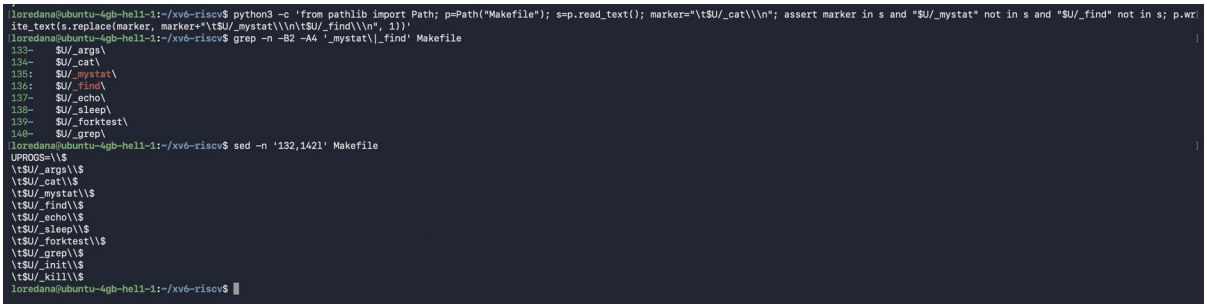


Figure 12: Makefile UPROGS entries for mystat and find

2.4 mystat and Link Semantics

mystat calls stat for every pathname and translates the numeric type to a readable label. README was a regular file at inode 2 with one link and a size of 2,441 bytes; . was directory inode 1; and console was device inode 27. After ln README r2, both names reported inode 2 and link count two. Removing README left r2 valid with link count one and unchanged contents.

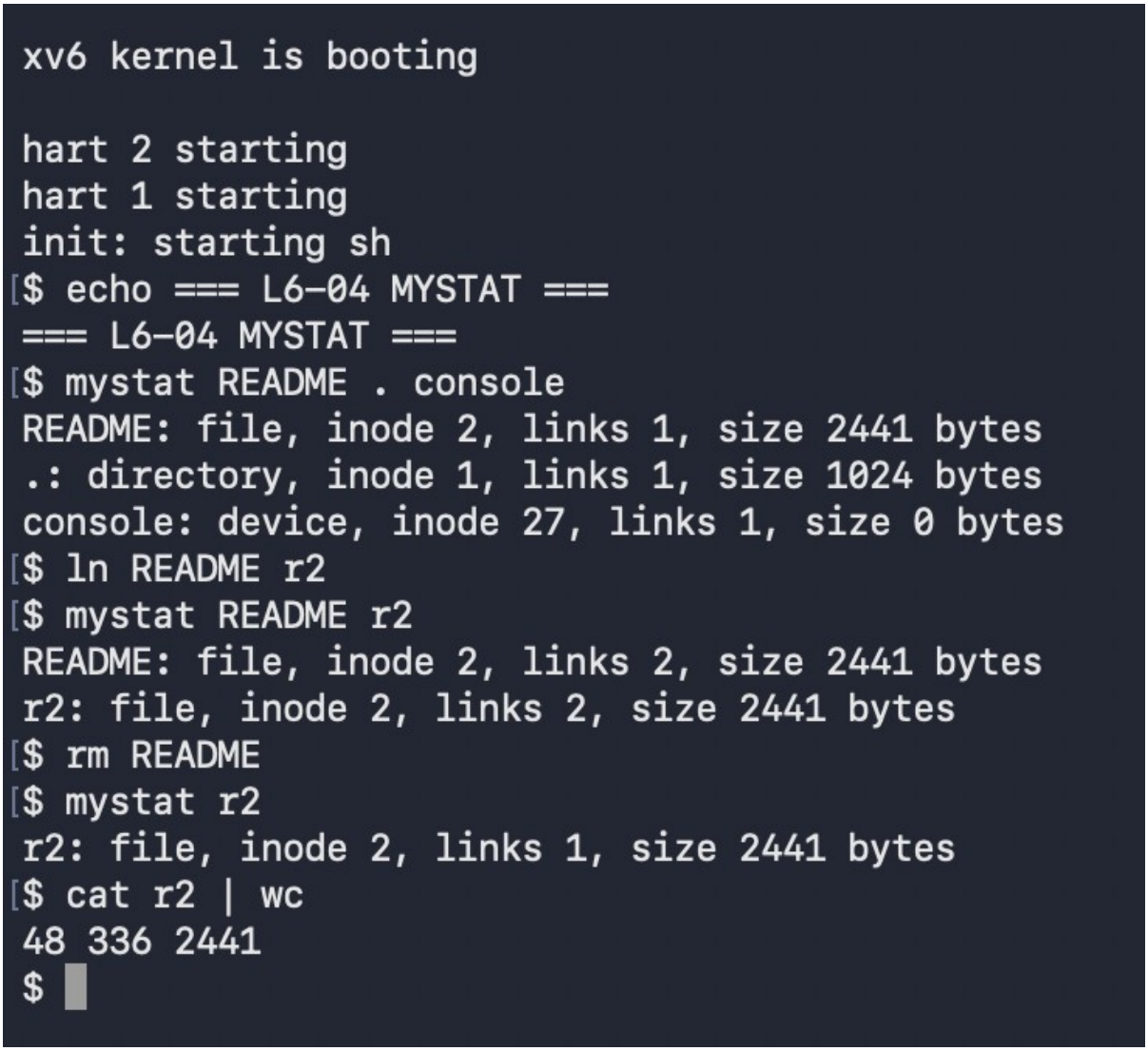


Figure 13: mystat output before and after linking and unlinking README

2.5 Unlink and Block Reclamation

sys_unlink removes the directory entry and decrements ip->nlink, persisting the new value with iupdate. For a directory, the parent's link count is also decremented. The data blocks are not necessarily freed at that instant: iput checks for the final in-memory reference to a valid inode whose nlink is zero, locks it, and calls itrunc to release its data blocks. This separation ensures that an open file remains usable after its final pathname has been removed.

```
[loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ printf '\n=== L6-05 UNLINK AND BLOCK FREEING ===\n'; grep -n -C10 'nlink--' kernel/sysfile.c

=== L6-05 UNLINK AND BLOCK FREEING ===
168-  }
169-  iunlockput(dp);
170-  iput(ip);
171-
172-  end_op();
173-
174-  return 0;
175-
176-bad:
177-  ilock(ip);
178:  ip->nlink--;
179-  iupdate(ip);
180-  iunlockput(ip);
181-  end_op();
182-  return -1;
183-}
184-
185-// Is the directory dp empty except for "." and ".." ?
186-static int
187-isdirempty(struct inode *dp)
188-{
--
229-    panic("unlink: nlink < 1");
230-  if (ip->type == T_DIR && !isdirempty(ip)) {
231-    iunlockput(ip);
232-    goto bad;
233-  }
234-
235-  memset(&de, 0, sizeof(de));
236-  if (writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
237-    panic("unlink: writei");
238-  if (ip->type == T_DIR) {
239:    dp->nlink--;
240-    iupdate(dp);
241-  }
242-  iunlockput(dp);
243-
244:  ip->nlink--;
245-  iupdate(ip);
246-  iunlockput(ip);
247-
248-  end_op();
249-
250-  return 0;
251-
252-bad:
253-  iunlockput(dp);
254-  end_op();
```

Figure 14: Link-count updates in the xv6 unlink system call

2.6 The Final iput Decision

The relevant condition is ip->ref == 1, ip->valid, and ip->nlink == 0. Only then can no other process still hold the cached inode. itrunc frees the file's direct and indirect data blocks, and the inode cache entry is invalidated. The same nlink == 0 state is also considered during orphan recovery.

```
[loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ grep -n -A18 'nlink == 0' kernel/fs.c
357:  int last = (ip->ref == 1 && ip->valid && ip->nlink == 0);
358-  uint dev = ip->dev, inum = ip->inum;
359-
360-  if (last) {
361-    // ip->ref == 1 means no other process can have ip locked.
362-    acquiresleep(&ip->lock);
363-    release(&itable.lock);
364-
365-    itrunc(ip); // free the data blocks (type stays nonzero on disk)
366-    ip->valid = 0;
367-
368-    releasesleep(&ip->lock);
369-
370-    acquire(&itable.lock);
371-  }
372-
373-  ip->ref--;
374-  release(&itable.lock);
375-
--
395:    if (dip->type != 0 && dip->nlink == 0) { // is an orphaned inode
396-      printk("ireclaim: orphaned inode %d\n", inum);
397-      ip = iget(dev, inum);
398-    }
399-    brelse(bp);
400-    if (ip) {
401-      begin_op();
402-      ilock(ip);
403-      iunlock(ip);
404-      iput(ip);
405-      end_op();
406-    }
407-  }
408-}
409-
410-// Inode content
411-//
412-// The content (data) associated with each inode is stored
413-// in blocks on the disk. The first NDIRECT block numbers
--
707:    if (ip->nlink == 0) {
708-      iunlockput(ip);
709-      return 0;
710-    }
711-    if (nameiparent && *path == '\0') {
712-      // Stop one level early.
713-      iunlock(ip);
714-      return ip;
715-    }
716-    if ((next = dirlookup(ip, name, 0)) == 0) {
717-      iunlockput(ip);
718-      return 0;
719-    }
720-    iunlockput(ip);
721-    ip = next;
722-  }
723-  if (nameiparent) {
724-    iput(ip);
725-    return 0;
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$
```

Figure 15: iput condition and itrunc-based block reclamation

2.7 File-System Constants and Disk Layout

xv6 uses 1,024-byte blocks, twelve direct block pointers, and 256 addresses in one indirect block. Therefore the largest file addressable by one inode is (12 + 256) × 1,024 = 274,432 bytes, or 268 KiB. Directory names are limited to DIRSIZ = 14 bytes.

The generated image reported 47 metadata blocks: one boot block, one superblock, 31 log blocks, 13 inode blocks, and one bitmap block. The image contained 1,953 data blocks and 2,000 blocks in total.

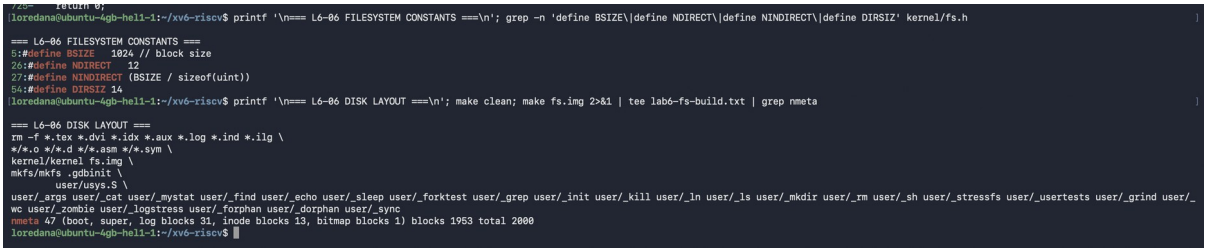


Figure 16: File-system constants and mkfs disk-layout summary

2.8 Name Truncation and Recursive find

A name longer than fourteen characters was stored as averyveryveryl, demonstrating the fixed-size xv6 directory name field. The recursive find implementation opens each directory, reads struct dirent records, skips unused entries plus . and .., constructs child paths, and visits them recursively. Searching from . found all three hard-linked names r2; searching from a for b also matched the directory itself after the name comparison was placed before the non-directory return.

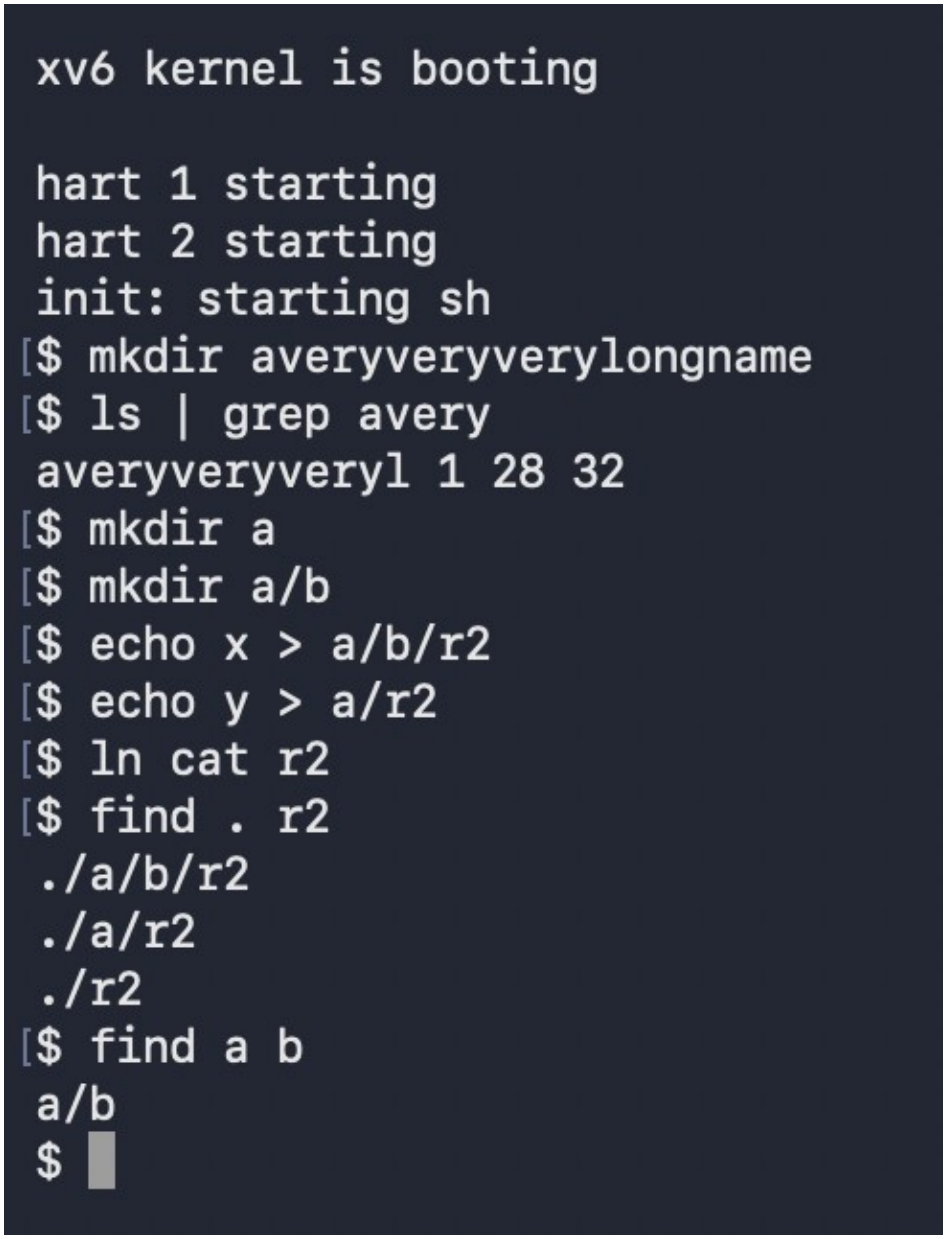


Figure 17: DIRSIZ truncation and recursive find results

2.9 Final Verification

The final run verified both programs in the rebuilt xv6 image. mystat correctly classified README, the current directory, and console, while find located ./README. The current directory showed link count three because the experiment had created subdirectory a, whose .. entry references the root directory.

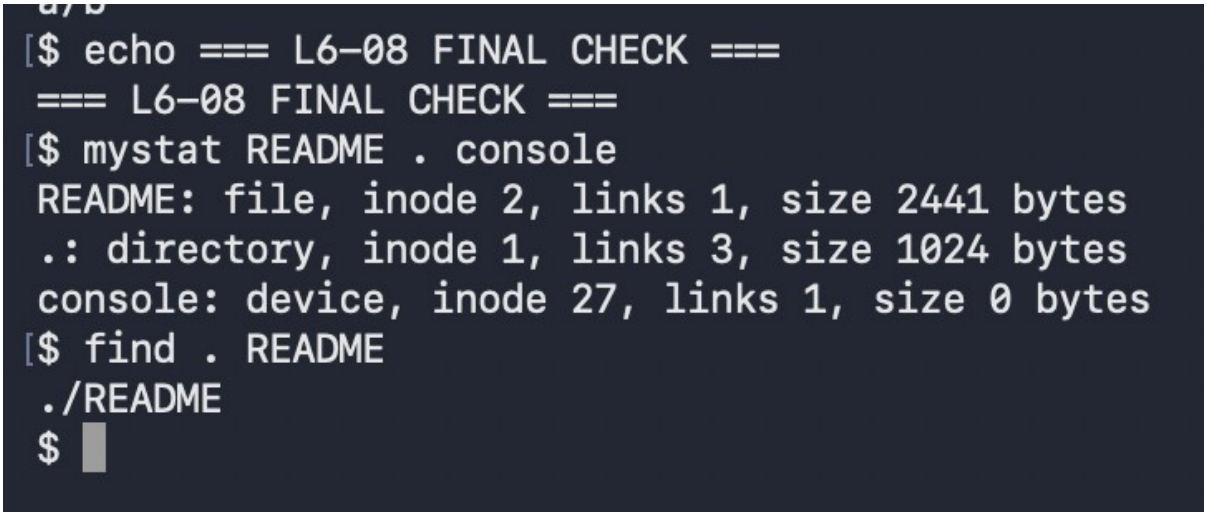


Figure 18: Final functional verification of mystat and find

Appendix A — user/mystat.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int type)
{
  switch(type){
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  default:
    return "unknown";
  }
}

int
main(int argc, char *argv[])
{
  struct stat st;

  if(argc < 2){
    fprintf(2, "usage: mystat path...\n");
    exit(1);
  }

  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }

    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }

  exit(0);
}
```

Appendix B — user/find.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"
#include "user/user.h"

static char *
basename(char *path)
{
  char *name = path + strlen(path);

  while(name > path && name[-1] != '/')
    name--;
  return name;
}

static void
find(char *start, char *target)
{
  char path[512];
  char name[DIRSIZ + 1];
  char *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(start, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", start);
    return;
  }

  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", start);
    close(fd);
    return;
  }

  if(strcmp(basename(start), target) == 0)
    printf("%s\n", start);

  if(st.type != T_DIR){
    close(fd);
    return;
  }

  if(strlen(start) + 1 + DIRSIZ + 1 > sizeof(path)){
    fprintf(2, "find: path too long %s\n", start);
    close(fd);
    return;
  }

  strcpy(path, start);
  p = path + strlen(path);
  if(p > path && p[-1] != '/')
    *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;

    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
      continue;

    memmove(p, name, DIRSIZ + 1);
    find(path, target);
  }

  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc != 3){
    fprintf(2, "usage: find <dir> <name>\n");
    exit(1);
  }

  find(argv[1], argv[2]);
  exit(0);
}
```

Conclusion and Critical Analysis

The two laboratory works connect the same abstraction at two levels. Linux presents files, directories, devices, and process information through a uniform pathname interface, while xv6 makes the underlying inode, directory-entry, block, and reference-count mechanisms small enough to trace directly. The experiments confirmed that a pathname is not the file itself: hard links share one inode, symbolic links contain another pathname, and unlink removes a name before it necessarily releases storage.

The permission experiments also showed why tests must be performed as an ordinary user. Owner identity chooses the owner mode bits but does not override them, directory read and execute permissions protect different operations, and special bits deliberately change deletion or execution semantics. The sparse-file experiment further demonstrated that logical length is not a reliable measure of physical consumption.

The most useful implementation lesson came from testing find with a directory as the target. The first version compared the basename only in the non-directory branch, so find a b traversed directory b but did not print it. Moving the comparison before the type test corrected the behavior without preventing recursion. This illustrates why a successful build is insufficient: tests must cover regular files, nested paths, hard-linked names, directories, and boundary conditions such as DIRSIZ.

Overall, the Linux observations and xv6 source inspection support one coherent model: directory entries provide names, inodes provide identity and metadata, link counts record persistent namespace references, open references preserve temporary access, and blocks are reclaimed only when both persistent and active references permit it. Both laboratory works were completed successfully, and the final xv6 image verified the required utilities.